

KCPC Week 3 Problems

The Lambda Calculus: Functions, Computation, and Numbers

This proof-based sheet follows the workshop slides. Attempt the five foundational warm-ups before the five linked problems; the final two programming exercises are extended homework. Work in the *untyped, pure* lambda calculus unless a question explicitly uses mathematical numbers as a teaching shorthand.

Conventions. Application associates left ($MNP = (MN)P$); abstraction extends as far right as possible ($\lambda xy.M = \lambda x.\lambda y.M$). $FV(M)$ is the set of free variables. $M[N/x]$ replaces free occurrences of x in M with N , renaming bound variables to avoid capture. A beta step is $(\lambda x.M)N \rightarrow_\beta M[N/x]$. Write $M \equiv_\alpha N$ for renaming of bound variables. Named closed terms (T , $\bar{n}$, etc.) below are abbreviations for their pure lambda definitions, not extra primitives. Show your reasoning, not just the final term.

1 Foundational warm-ups

1. Terms and parentheses

Prerequisites: Variables, abstraction, application. Expand $\lambda x.\lambda y.x(yz)$ into the fully parenthesised grammar $M ::= x \mid (\lambda x.M) \mid (MM)$. Identify every free variable and prove your answer using the recursive rules for FV . Is $\lambda x.xy$ an abstraction or an application at its root?

2. Free variables and substitution

Prerequisites: Problem 1. Define $FV(x)$, $FV(MN)$, and $FV(\lambda x.M)$. Derive a formula for $FV((\lambda x.M)N)$ in terms of $FV(M)$ and $FV(N)$; evaluate it for $(\lambda x.xy)z$. Explain why a bound occurrence of x is not a free occurrence.

3. Alpha conversion and capture

Prerequisites: Problems 1–2. Prove $\lambda x.\lambda y.xy \equiv_\alpha \lambda u.\lambda v.uv$ but not $\lambda x.\lambda y.xy \equiv_\alpha \lambda y.\lambda y.yy$. Reduce $(\lambda x.\lambda y.x)y$ correctly in one beta step and explain why $\lambda y.y$ would be the wrong result.

4. Identity and constant functions

Prerequisites: Beta reduction. Let $I = \lambda x.x$, $K = \lambda x.\lambda y.x$, and $F = \lambda x.\lambda y.y$. Prove, for arbitrary terms A, B , that $IA \rightarrow_\beta A$, $KAB \rightarrow_\beta^* A$, and $FAB \rightarrow_\beta^* B$. Compare the number of beta steps required for the last two. Give the two-step trace for KII .

5. Currying and composition

Prerequisites: Problem 4. Set $C = \lambda f.\lambda g.\lambda x.f(gx)$. Prove $Cfga \rightarrow_\beta^* f(ga)$ for arbitrary f, g, a ; explain why C is a sequence of one-input functions rather than a three-input primitive. Then prove $CII \equiv_\beta I$ (beta-convertible, not necessarily one beta step).

2 Linked ideas: proofs and encodings

6. Boolean selectors

Prerequisites: Problems 3–5; currying. Take $T = K$ and $F = F$. Define pure closed terms AND, OR (two Boolean arguments each), and NOT (one Boolean argument). Prove all four rows of the truth table for each binary operation by using the selector equations, and prove both cases for NOT. Why is choosing between two arbitrary terms possible without a built-in conditional?

7. Church numerals and successor

Prerequisites: Problems 4–6; induction. Define $\bar{n} = \lambda f.\lambda x.f^n(x)$, where $f^0(x) = x$ and $f^{n+1}(x) = f(f^n(x))$. Write $\bar{0}$, $\bar{1}$, and $\bar{2}$ as closed terms. Construct a pure SUCC and prove for every $n \geq 0$ that $\text{SUCC } \bar{n} \equiv_\beta \overline{n+1}$. Compute $\bar{2}ga$ by beta reduction.

8. Arithmetic as iteration

Prerequisites: Problem 7; composition. Construct pure ADD and MUL with the expected effects $\text{ADD } \bar{m} \bar{n} \equiv_\beta \overline{m+n}$ and $\text{MUL } \bar{m} \bar{n} \equiv_\beta \overline{mn}$. Prove the claims using $f^m(f^n(x)) = f^{m+n}(x)$ and $(f^n)^m(x) = f^{mn}(x)$, not just examples. Explain why $\bar{n} \bar{m} f x$ behaves like $f^{mn}(x)$ (including $0^0 = 1$), and give a fully eta-expanded POW term beta-convertible to $\overline{m^n}$.

9. Subtraction by state threading

Prerequisites: Problems 6–8; induction on an iterator. Let $\langle a, b \rangle = \lambda s.sab$, $\text{FIRST} = \lambda p.pT$, $\text{SECOND} = \lambda p.pF$, and $\text{PAIR} = \lambda a.\lambda b.\lambda s.sab$. Define

$$\text{STEP} = \lambda p.\text{PAIR} (\text{SECOND } p) (\text{SUCC}(\text{SECOND } p)).$$

Prove by induction that $\text{STEP}^n \langle \bar{0}, \bar{0} \rangle \equiv_\beta \langle \overline{\max(n-1, 0)}, \bar{n} \rangle$. Check the base case and the step from $n = 0$ separately. Use the result to define a predecessor and truncated subtraction $\text{SUB } m n = \overline{\max(m-n, 0)}$. State explicitly what happens at zero.

10. Logic meets numbers

Prerequisites: Problems 6–9. Find a closed term ISZERO such that $\text{ISZERO } \bar{0} \equiv_{\beta} \text{T}$ and $\text{ISZERO } \overline{n+1} \equiv_{\beta} \text{F}$. Prove this for all n . Define EVEN using NOT and iteration, then prove $\text{EVEN } \bar{n}$ selects T exactly when n is even. Explain how $\text{ISZERO (SUB } m \ n)$ implements the comparison $m \leq n$ for Church numerals.

Optional slide challenge: Divide Church naturals by iterating a pair of quotient and remainder m times for a nonzero divisor d . Specify the result for $d = 0$.

3 Extended homework (C++ required)

11. Church arithmetic on a stack

Prerequisites: Problems 7–8; modular arithmetic and a stack. Input is an integer q ($1 \leq q \leq 200000$) followed by q whitespace-separated postfix tokens: Z pushes $\bar{0}$; S replaces the top numeral by its successor; A and M pop a then b and push $\text{ADD } ba$ or $\text{MUL } ba$. The input is guaranteed well-formed, leaving one numeral. Output its value modulo $10^9 + 7$. Prove that maintaining a residue per stack item returns the interpretation of the encoded term. Implement in C++ (optional: also Python). Do not expand the lambda term into a string.

12. Typed postfix booleans and numbers

Prerequisites: Problems 6–11; invariants and induction. Extend the preceding token language with T , F (push Boolean selectors), N (NOT), D (AND), R (OR), Z? (ISZERO), and I (IF). For I , pop number b , then number a , then Boolean c , and push cab . Assume the postfix program is well-typed and ends with one numeral. Output its value modulo $10^9 + 7$. Since $\overline{10^9 + 7}$ has residue zero but is not zero, explain how to represent a numeric stack entry so Z? stays correct. Prove the extended interpreter correct by induction on tokens and implement it in C++.